Solo E-Commerce Platform — Low Level Design (LLD)

Document Version: 1.0
Date: 17 March 2026
Author: Solo Engineering Team
Status: Final

---

1. Purpose

This Low Level Design document specifies the internal structure, class design, data models, algorithms, and API contracts for every module of the Solo E-Commerce platform. It translates the High Level Design into implementation-level detail.

---

2. Backend Module Design

2.1 Application Bootstrap (`main.ts`)

```
NestFactory.create(AppModule)
  │
  ├── Global Prefix: /api
  ├── Helmet: Security headers
  ├── CORS: { origin: FRONTEND_URL }
  ├── ValidationPipe: { whitelist: true, transform: true }
  ├── ThrottlerGuard: 1000 requests / 60 seconds
  ├── Static Assets: /uploads → ServeStaticModule
  └── Listen: PORT (default 3000)
```

2.2 Module Dependency Graph

```
AppModule
├── ConfigModule (global)
├── ThrottlerModule (global)
├── ServeStaticModule (/uploads)
├── PrismaModule (global)
│
├── AuthModule
│   ├── UsersModule (forwardRef)
│   ├── JwtModule (access + refresh)
│   ├── PassportModule
│   ├── JwtStrategy
│   └── LocalStrategy
│
├── UsersModule
│   ├── PrismaService
│   ├── LoyaltyService
│   └── PaymentMethodsService
│
├── ProductsModule
│   ├── PrismaService
│   └── ConfigService
│
├── CategoriesModule → PrismaService
├── BrandsModule → PrismaService
├── CartModule → ProductsModule
├── OrdersModule → CartModule, ProductsModule, InvoiceService
├── FavoritesModule → PrismaService
│
├── AdminModule → PrismaService, OrdersModule
│   └── ReportsService
│
├── PromosModule → PrismaService
├── StripeModule → ConfigService
├── MediaModule → MulterModule
├── ContentModule → PrismaService
├── CmsModule → PrismaService
├── CustomersModule → PrismaService
├── BlogModule → PrismaService
├── NavigationModule → PrismaService
├── CollectionsModule → PrismaService
├── SettingsModule → PrismaService
└── PackagesModule → PrismaService
```

---

3. Authentication Module — Detailed Design

3.1 Class Diagram

```
AuthController
   │
   ├── register(dto: RegisterDto): { user, tokens }
   ├── login(dto: LoginDto): { user, tokens }
   ├── refresh(dto: RefreshTokenDto): { accessToken }
   ├── logout(dto: LogoutDto): void
   ├── getMe(@CurrentUser): User
   ├── changePassword(dto: ChangePasswordDto): void
   ├── forgotPassword(dto: ForgotPasswordDto): void
   ├── resetPassword(dto: ResetPasswordDto): void
   ├── verifyEmail(dto: VerifyEmailDto): void
   └── resendVerification(dto: ResendVerificationDto): void
         │
         ▼
AuthService
   │
   ├── register(dto): User + Tokens
   │     ├── Check email uniqueness
   │     ├── Hash password (Argon2id)
   │     ├── Create user record
   │     ├── Create loyalty wallet
   │     ├── Generate email verification token
   │     ├── Send verification email
   │     └── Generate JWT tokens
   │
   ├── login(email, password): User + Tokens
   │     ├── Find user by email
   │     ├── Verify password (Argon2id)
   │     ├── Check email verified status
   │     └── Generate JWT tokens
   │
   ├── refresh(refreshToken): AccessToken
   │     ├── Find refresh token in DB
   │     ├── Check not expired/revoked
   │     └── Generate new access token
   │
   ├── generateTokens(userId, email, role):
   │     ├── Access Token: { sub, email, role } → 15min
   │     └── Refresh Token: random UUID → 7 days → stored in DB
   │
   └── validateUser(payload): User
         └── Find user by ID from JWT sub claim
```

3.2 Token Data Structures

```
AccessToken JWT Payload:
{
  sub: string      // User ID (UUID)
  email: string    // User email
  role: string     // CUSTOMER | ADMIN | SUPER_ADMIN
  iat: number      // Issued at (Unix timestamp)
  exp: number      // Expires at (iat + 15 minutes)
}

RefreshToken DB Record:
{
  id: UUID
  token: string        // Random UUID
  userId: string       // FK → User
  expiresAt: DateTime  // Now + 7 days
  isRevoked: boolean   // Set true on logout
  createdAt: DateTime
}
```

3.3 Password Hashing

```
Algorithm: Argon2id
   ├── Memory Cost: 65536 KB (64 MB)
   ├── Time Cost: 3 iterations
   ├── Parallelism: 4
   └── Salt: Auto-generated (16 bytes)

Hash Format: $argon2id$v=19$m=65536,t=3,p=4$<salt>$<hash>
```

3.4 Rate Limiting (Auth Endpoints)

Endpoint	Limit	Window
POST /auth/register	3 requests	1 hour
POST /auth/login	5 requests	15 minutes
POST /auth/forgot-password	3 requests	1 hour
POST /auth/reset-password	5 requests	15 minutes
All other endpoints	1000 requests	1 minute

---

4. Products Module — Detailed Design

4.1 Class Diagram

```
ProductsController
   │
   ├── findAll(filters: ProductFilterDto): PaginatedResponse<Product[]>
   ├── getFeatured(): Product[]
   ├── getBestSellers(): Product[]
   ├── getNewArrivals(): Product[]
   ├── findOne(slugOrId: string): Product
   ├── getRelated(id: string): Product[]
   ├── create(dto: CreateProductDto): Product          [ADMIN]
   ├── update(id: string, dto: UpdateProductDto): Product [ADMIN]
   └── delete(id: string): void                         [ADMIN]
         │
         ▼
ProductsService
   │
   ├── findAll(filters):
   │     ├── Build Prisma where clause from filters
   │     │     ├── category → categories.name contains
   │     │     ├── brand → brands.name contains
   │     │     ├── minPrice/maxPrice → product_pricing.selling_price range
   │     │     ├── search → name ILIKE or description ILIKE
   │     │     └── subcategoryId → subcategory FK
   │     ├── Apply pagination (skip, take)
   │     ├── Include: images, pricing, category, brand
   │     ├── Call resolveProductImageUrls(products)
   │     └── Transform to response format
   │
   ├── resolveProductImageUrls(products[]):
   │     ├── Collect all media_asset_id values from images
   │     ├── Batch query: SELECT id, key FROM media_assets WHERE id IN (...)
   │     ├── Build URL map: { uuid → uploadsBaseUrl/key }
   │     └── Mutate image.url in-place with resolved URLs
   │
   └── transformProduct(raw):
         ├── Map DB fields to API response format
         ├── Calculate display price (with overrides)
         ├── Build image URLs array
         └── Return clean JSON structure
```

4.2 Product Filter DTO

```
class ProductFilterDto {
  @IsOptional() @IsString()     category?: string
  @IsOptional() @IsString()     brand?: string
  @IsOptional() @IsString()     subcategoryId?: string
  @IsOptional() @Type(() => Number) @IsNumber()  minPrice?: number
  @IsOptional() @Type(() => Number) @IsNumber()  maxPrice?: number
  @IsOptional() @IsString()     search?: string
  @IsOptional() @Type(() => Number) @IsInt() @Min(1)    page?: number = 1
  @IsOptional() @Type(() => Number) @IsInt() @Min(1)    limit?: number = 20
  @IsOptional() @IsString()     sortBy?: string
  @IsOptional() @IsString()     sortOrder?: 'asc' | 'desc'
}
```

4.3 Product Response Structure

```
{
  "id": "uuid",
  "name": "Eva Solo Café Latte Tumbler",
  "slug": "eva-solo-cafe-latte-tumbler",
  "brand": "Eva Solo",
  "category": "Tea & Coffee",
  "subcategory": "Tumblers",
  "description": "...",
  "shortDescription": "...",
  "price": 200.00,
  "originalPrice": 250.00,
  "currency": "AED",
  "images": [
    { "url": "https://host/uploads/products/img1.jpg", "displayOrder": 1 }
  ],
  "isNew": true,
  "isFeatured": false,
  "isBestSeller": false,
  "inStock": true,
  "specifications": [
    { "key": "Material", "value": "Borosilicate Glass" }
  ],
  "dimensions": { "width": 8, "height": 12, "depth": 8, "unit": "cm" },
  "metaTitle": "...",
  "metaDescription": "..."
}
```

---

5. Cart Module — Detailed Design

5.1 Class Diagram

```
CartController
   │
   ├── getCart(@CurrentUser): CartResponse
   ├── addItem(@CurrentUser, dto: AddCartItemDto): CartResponse
   ├── updateItem(@CurrentUser, itemId, dto: UpdateCartItemDto): CartResponse
   ├── removeItem(@CurrentUser, itemId): CartResponse
   └── clearCart(@CurrentUser): void
         │
         ▼
CartService
   │
   ├── getOrCreateCart(userId):
   │     ├── Find cart WHERE userId = ?
   │     ├── If not found → Create empty cart
   │     └── Return cart with items
   │
   ├── addItem(userId, dto):
   │     ├── Get/create cart
   │     ├── Check if product already in cart
   │     │     ├── YES → Increment quantity
   │     │     └── NO → Create new CartItem
   │     ├── Validate stock availability
   │     └── Return enriched cart
   │
   ├── enrichCartItemsWithProducts(items):
   │     ├── Extract unique productIds from items
   │     ├── Batch fetch products with pricing + images
   │     ├── Call productsService.resolveProductImageUrls()
   │     ├── Map items with product details
   │     └── Calculate line totals
   │
   └── calculateSummary(items, promoCode?):
         ├── Subtotal = Σ (item.price × item.quantity)
         ├── Discount = apply promo code if valid
         ├── VAT = (Subtotal - Discount) × 0.05
         ├── Shipping = calculate based on rules
         └── Total = Subtotal - Discount + VAT + Shipping
```

5.2 Cart Response Structure

```
{
  "id": "cart-uuid",
  "items": [
    {
      "id": "cart-item-uuid",
      "productId": "product-uuid",
      "name": "Eva Solo Café Latte Tumbler",
      "price": 200.00,
      "quantity": 2,
      "imageUrl": "https://host/uploads/products/img.jpg",
      "lineTotal": 400.00
    }
  ],
  "summary": {
    "subtotal": 400.00,
    "discount": 0.00,
    "vat": 20.00,
    "shipping": 0.00,
    "total": 420.00,
    "itemCount": 2
  },
  "promoCode": null
}
```

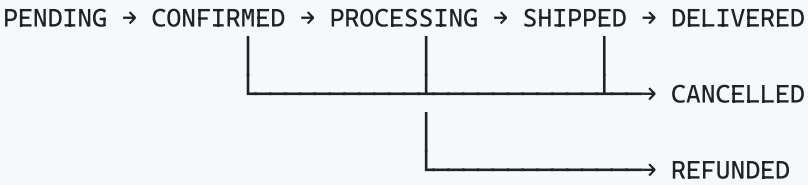
---

6. Orders Module — Detailed Design

6.1 Order Creation Flow

```
POST /api/orders (CreateOrderDto)
      │
      ▼
OrdersService.create(userId, dto)
      │
      ├── 1. Validate cart is not empty
      ├── 2. Validate shipping address belongs to user
      ├── 3. Validate billing address belongs to user
      ├── 4. Validate promo code (if provided)
      ├── 5. Verify payment with Stripe
      │       └── stripe.paymentIntents.retrieve(paymentIntentId)
      │
      ├── 6. Create Order record:
      │       ├── orderNumber: "SO-" + timestamp + random
      │       ├── status: PENDING → CONFIRMED
      │       ├── items: snapshot of cart items with prices
      │       ├── pricing: subtotal, discount, vat, shipping, total
      │       ├── addresses: shipping + billing snapshots
      │       └── paymentIntentId: Stripe reference
      │
      ├── 7. Deduct promo code usage count
      ├── 8. Award loyalty points (1 point per AED spent)
      ├── 9. Clear user's cart
      ├── 10. Create OrderStatusHistory entry
      ├── 11. Send order confirmation email
      └── 12. Return order details
```

6.2 Order Status State Machine



Status	Description	Triggered By
PENDING	Order created, payment pending	System (auto)
CONFIRMED	Payment verified	System (auto)
PROCESSING	Being prepared for shipping	Admin (manual)
SHIPPED	Dispatched with tracking number	Admin (manual)
DELIVERED	Received by customer	Admin (manual)
CANCELLED	Order cancelled	Admin or Customer

Status	Description	Triggered By
REFUNDED	Payment refunded	Admin (manual)

6.3 Invoice Generation

```
InvoiceService.generatePdf(orderId):
  ├── Fetch order with items, addresses, user
  ├── Build PDF layout:
  │    ├── Company header (Solo logo, address, TRN)
  │    ├── Invoice number + date
  │    ├── Customer details
  │    ├── Item table (name, qty, unit price, total)
  │    ├── Summary (subtotal, discount, VAT 5%, total)
  │    └── Footer (terms, bank details)
  ├── Generate PDF buffer
  └── Return as stream / save to filesystem
```

---

7. Admin Module — Detailed Design

7.1 Dashboard Statistics

```
AdminService.getDashboardStats():
   │
   ├── Total Revenue (this month):
   │   └── SELECT SUM(total) FROM orders WHERE status != CANCELLED AND createdAt >= sta
   │
   ├── Total Orders (this month):
   │   └── SELECT COUNT(*) FROM orders WHERE createdAt >= startOfMonth
   │
   ├── New Customers (this month):
   │   └── SELECT COUNT(*) FROM users WHERE role = CUSTOMER AND createdAt >= startOfMon
   │
   ├── Average Order Value:
   │   └── Total Revenue / Total Orders
   │
   ├── Top Selling Products (top 10):
   │   └── SELECT productId, SUM(quantity) as totalSold
   │       FROM order_items GROUP BY productId
   │       ORDER BY totalSold DESC LIMIT 10
   │
   ├── Recent Orders (last 10):
   │   └── SELECT * FROM orders ORDER BY createdAt DESC LIMIT 10
   │
   ├── Revenue by Category:
   │   └── JOIN order_items → products → categories, aggregate by category
   │
   └── Order Status Distribution:
       └── SELECT status, COUNT(*) FROM orders GROUP BY status
```

7.2 Reports Service

```
ReportsService
   │
   ├── getFinancialReport(dateRange):
   │     ├── Revenue by day/week/month
   │     ├── Cost of goods sold (if cost price available)
   │     ├── Gross margin
   │     └── VAT collected
   │
   ├── getOrdersReport(dateRange):
   │     ├── Orders by status
   │     ├── Orders by day
   │     ├── Average processing time
   │     └── Cancellation rate
   │
   ├── getProductsReport():
   │     ├── Top sellers by revenue
   │     ├── Top sellers by quantity
   │     ├── Low stock alerts
   │     └── Products with no sales
   │
   ├── getCustomersReport():
   │     ├── New vs returning customers
   │     ├── Top customers by spend
   │     ├── Customer lifetime value
   │     └── Registration trend
   │
   └── getVatReport(dateRange):
         ├── Total taxable amount
         ├── VAT collected
         ├── VAT rate applied (5%)
         └── Breakdown by order
```

---

8. Content/CMS Module — Detailed Design

8.1 Homepage Structure

```
HomePageConfig
   │
   ├── HomePageSection[]
   │     ├── id: UUID
   │     ├── type: HERO_BANNER | FEATURED_PRODUCTS | CATEGORY_TILES |
   │     │         BEST_SELLERS | NEW_ARRIVALS | PROMO_BANNER | CUSTOM_HTML
   │     ├── title: string
   │     ├── config: JSON (section-specific configuration)
   │     ├── displayOrder: number
   │     └── isActive: boolean
   │
   └── Banner[]
         ├── id: UUID
         ├── title: string
         ├── subtitle: string
         ├── ctaText: string (button label)
         ├── ctaUrl: string (button link)
         ├── imageDesktopUrl: string
         ├── imageMobileUrl: string
         ├── placement: HOME_HERO | HOME_TOP | HOME_MID | HOME_BOTTOM | CATEGORY_TOP | CA
         ├── displayOrder: number
         ├── isActive: boolean
         ├── startDate: DateTime?
         └── endDate: DateTime?
```

8.2 Banner Placement Flow

```
GET /api/content/banners?placement=HOME_HERO
   │
   └── ContentService.getActiveBanners(placement):
         ├── WHERE placement = ?
         ├── AND isActive = true
         ├── AND (startDate IS NULL OR startDate <= NOW())
         ├── AND (endDate IS NULL OR endDate >= NOW())
         ├── ORDER BY displayOrder ASC
         └── Return Banner[]
```

---

9. Stripe Payment Module — Detailed Design

9.1 Payment Flow

9.2 Payment Amount Calculation

```
Amount (in fils — Stripe uses smallest currency unit):
  amount = Math.round(total × 100)  // AED 420.00 → 42000 fils
  currency = "aed"
```

---

10. Media Upload Module — Detailed Design

10.1 Upload Pipeline

```
POST /api/media/upload (multipart/form-data)
   │
   ├── Multer Middleware:
   │     ├── File size limit: 5 MB
   │     ├── Allowed MIME types: image/jpeg, image/png, image/webp, image/svg+xml
   │     ├── Storage: disk storage
   │     └── Filename: {timestamp}-{random}-{original}
   │
   ├── MediaService.upload(file, folder, optimize):
   │     ├── Destination: /uploads/{folder}/{filename}
   │     ├── If optimize: resize image (sharp.js)
   │     │     ├── Max width: 1920px
   │     │     ├── Format: WebP (if not SVG)
   │     │     └── Quality: 85%
   │     ├── Create media_assets record:
   │     │     ├── id: UUID
   │     │     ├── key: {folder}/{filename}
   │     │     ├── mimeType: detected
   │     │     ├── size: file bytes
   │     │     └── createdAt: now
   │     └── Return { url, id, key }
   │
   └── Response:
       {
         "url": "http://host/uploads/products/1679234567-abc.webp",
         "id": "media-asset-uuid",
         "key": "products/1679234567-abc.webp"
       }
```

---

11. Favorites Module — Detailed Design

11.1 Data Model

```
Favorite {
  id: UUID (PK)
  userId: UUID (FK → User)
  productId: UUID (FK → Product)
  createdAt: DateTime

  @@unique([userId, productId])  // Prevent duplicates
}
```

11.2 Toggle Logic

```
POST /api/favorites/{productId}/toggle
  │
  └── FavoritesService.toggle(userId, productId):
      ├── Find existing: WHERE userId = ? AND productId = ?
      ├── If exists → DELETE → return { isFavorite: false }
      └── If not exists → CREATE → return { isFavorite: true }
```

---

12. Promo Code Module — Detailed Design

12.1 Promo Code Validation

```
POST /api/promo-codes/validate { code, subtotal }
  │
  └── PromosService.validate(code, subtotal, userId):
      │
      ├── 1. Find promo code (case-insensitive)
      ├── 2. Check isActive = true
      ├── 3. Check startDate <= now <= endDate
      ├── 4. Check usageCount < usageLimit
      ├── 5. Check user hasn't already used it (if single-use per user)
      ├── 6. Check minimumOrderAmount <= subtotal
      │
      ├── Calculate discount:
      │   ├── PERCENTAGE: subtotal × (percentage / 100), cap at maxDiscount
      │   ├── FIXED_AMOUNT: fixedAmount (if subtotal >= minimum)
      │   └── FREE_SHIPPING: shipping cost becomes 0
      │
      └── Return { valid: true, discount, type, message }
```

12.2 Promo Code Data Model

```
PromoCode {
  id: UUID
  code: string (unique, uppercase)
  type: PERCENTAGE | FIXED_AMOUNT | FREE_SHIPPING
  value: Decimal (percentage or fixed amount)
  minimumOrderAmount: Decimal?
  maxDiscount: Decimal? (cap for percentage)
  usageLimit: Int?
  usageCount: Int (default 0)
  perUserLimit: Int? (default 1)
  startDate: DateTime
  endDate: DateTime
  isActive: Boolean
  createdAt: DateTime
  updatedAt: DateTime
}
```

---

13. Loyalty Module — Detailed Design

13.1 Points Calculation

```
Points Earned = floor(orderTotal in AED)
  Example: AED 420.00 → 420 loyalty points

Points Redemption Rate: 100 points = AED 1.00
  Example: 500 points = AED 5.00 discount
```

13.2 Transaction Types

```
LoyaltyTransaction {
  id: UUID
  walletId: UUID (FK → LoyaltyWallet)
  type: EARNED | REDEEMED | ADJUSTMENT | EXPIRED
  points: Int (positive for earn, negative for redeem)
  description: string
  orderId: UUID? (null for adjustments)
  createdAt: DateTime
}
```

---

14. Frontend Provider Design

14.1 Provider Architecture

```
MultiProvider (main.dart)
   │
   ├── AuthProvider (ChangeNotifier)
   │     ├── State: user, isAuthenticated, isLoading, tokens
   │     ├── Methods: login(), register(), logout(), refreshToken()
   │     └── Persistence: flutter_secure_storage (tokens)
   │
   ├── CartProvider (ChangeNotifier)
   │     ├── State: cart, items[], summary, isLoading
   │     ├── Methods: loadCart(), addItem(), updateQuantity(), removeItem(), clear()
   │     └── Sync: Server-synced via CartApi
   │
   ├── ProductListProvider (ChangeNotifier)
   │     ├── State: products[], isLoading, hasMore, filters
   │     ├── Methods: loadProducts(), loadMore(), applyFilters(), reset()
   │     └── Pagination: Cursor-based, 20 items per page
   │
   ├── ProductDetailsProvider (ChangeNotifier)
   │     ├── State: product, isLoading, selectedVariant, selectedImage
   │     └── Methods: loadProduct(id), selectVariant(), selectImage()
   │
   ├── CategoryProvider (ChangeNotifier)
   │     ├── State: categories[], isLoading
   │     └── Methods: loadCategories()
   │
   ├── FavoritesProvider (ChangeNotifier)
   │     ├── State: favoriteIds Set<String>, isLoading
   │     ├── Methods: loadFavorites(), toggle(productId), isFavorite(id)
   │     └── Sync: Server-synced via FavoritesApi
   │
   ├── SearchProvider (ChangeNotifier)
   │     ├── State: results[], query, isLoading
   │     └── Methods: search(query), clear()
   │
   ├── HomeCmsProvider (ChangeNotifier)
   │     ├── State: sections[], banners[], isLoading
   │     ├── Methods: loadHomeCms()
   │     └── Guard: Prevents concurrent duplicate loads
   │
   ├── HomeProvider (ChangeNotifier)
   │     ├── State: featured[], bestSellers[], newArrivals[]
   │     └── Methods: loadHomeData()
   │
   ├── AccountProvider (ChangeNotifier)
   │     ├── State: profile, addresses[], orders[], loyalty
   │     └── Methods: loadProfile(), updateProfile(), manageAddresses()
   │
   └── ContentProvider (ChangeNotifier)
```

```
        ├── State: banners[], landingPages
        └── Methods: loadBanners(), loadLandingPage(slug)
```

14.2 API Client Design

```
ApiClient (lib/core/api/)
    │
    ├── baseUrl: "http://localhost:3000/api"
    │
    ├── Interceptors:
    │   ├── AuthInterceptor:
    │   │   ├── Attach Bearer token to every request
    │   │   ├── On 401 → attempt token refresh
    │   │   └── On refresh fail → force logout
    │   │
    │   └── ErrorInterceptor:
    │       ├── Parse error response
    │       └── Throw typed exceptions
    │
    ├── Methods:
    │   ├── get<T>(path, {queryParams}): T
    │   ├── post<T>(path, body): T
    │   ├── patch<T>(path, body): T
    │   ├── delete(path): void
    │   └── multipart(path, file, fields): T
    │
    └── Token Storage:
        ├── accessToken → flutter_secure_storage
        └── refreshToken → flutter_secure_storage
```

---

15. Database Schema — Key Tables

15.1 Core Tables with Column Detail

```
-- Users Table
users (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email           VARCHAR(255) UNIQUE NOT NULL,
  password_hash   TEXT NOT NULL,
  first_name      VARCHAR(100),
  last_name       VARCHAR(100),
  phone           VARCHAR(20),
  role            user_role DEFAULT 'CUSTOMER',  -- CUSTOMER | ADMIN | SUPER_ADMIN
  is_active       BOOLEAN DEFAULT true,
  email_verified  BOOLEAN DEFAULT false,
  created_at      TIMESTAMPTZ DEFAULT NOW(),
  updated_at      TIMESTAMPTZ DEFAULT NOW()
)

-- Products Table
products (
  id              UUID PRIMARY KEY,
  name            VARCHAR(500) NOT NULL,
  slug            VARCHAR(500) UNIQUE,
  description     TEXT,
  short_desc      TEXT,
  brand_id        UUID REFERENCES brands(id),
  category_id     UUID REFERENCES categories(id),
  subcategory_id  UUID REFERENCES subcategories(id),
  designer_id     UUID REFERENCES designers(id),
  country_id      UUID REFERENCES countries(id),
  is_active       BOOLEAN DEFAULT true,
  is_featured     BOOLEAN DEFAULT false,
  is_best_seller  BOOLEAN DEFAULT false,
  is_new          BOOLEAN DEFAULT false,
  created_at      TIMESTAMPTZ DEFAULT NOW(),
  updated_at      TIMESTAMPTZ DEFAULT NOW()
)

-- Product Pricing
product_pricing (
  id              UUID PRIMARY KEY,
  product_id      UUID REFERENCES products(id),
  selling_price   DECIMAL(10,2) NOT NULL,
  original_price  DECIMAL(10,2),
  cost_price      DECIMAL(10,2),
  currency        VARCHAR(3) DEFAULT 'AED',
  vat_inclusive    BOOLEAN DEFAULT true,
  vat_rate        DECIMAL(5,2) DEFAULT 5.00
)

-- Orders Table
orders (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  order_number      VARCHAR(50) UNIQUE NOT NULL,
  user_id           UUID REFERENCES users(id),
  status            order_status DEFAULT 'PENDING',
  subtotal          DECIMAL(12,2) NOT NULL,
  discount          DECIMAL(12,2) DEFAULT 0,
  vat_amount        DECIMAL(12,2) NOT NULL,
  shipping_amount   DECIMAL(12,2) DEFAULT 0,
  total             DECIMAL(12,2) NOT NULL,
  payment_intent_id VARCHAR(255),
  payment_status    VARCHAR(50),
  promo_code_id     UUID REFERENCES promo_codes(id),
  loyalty_earned    INT DEFAULT 0,
  loyalty_redeemed  INT DEFAULT 0,
  shipping_address  JSONB NOT NULL,
  billing_address   JSONB NOT NULL,
  notes             TEXT,
  created_at        TIMESTAMPTZ DEFAULT NOW(),
  updated_at        TIMESTAMPTZ DEFAULT NOW()
)

-- Order Items
order_items (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id        UUID REFERENCES orders(id),
  product_id      UUID,
  product_name    VARCHAR(500) NOT NULL,
  product_image   TEXT,
  quantity        INT NOT NULL,
  unit_price      DECIMAL(10,2) NOT NULL,
  total_price     DECIMAL(10,2) NOT NULL,
  created_at      TIMESTAMPTZ DEFAULT NOW()
)
```

15.2 Indexes

```
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_products_category ON products(category_id);
CREATE INDEX idx_products_brand ON products(brand_id);
CREATE INDEX idx_products_slug ON products(slug);
CREATE INDEX idx_orders_user ON orders(user_id);
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_orders_created ON orders(created_at);
CREATE INDEX idx_order_items_order ON order_items(order_id);
CREATE INDEX idx_cart_user ON carts(user_id);
CREATE INDEX idx_favorites_user_product ON favorites(user_id, product_id);
CREATE INDEX idx_refresh_tokens_token ON refresh_tokens(token);
```

---

16. Error Handling Strategy

16.1 Backend Error Hierarchy

```
HttpException (NestJS)
  ├── BadRequestException (400) — Invalid input, validation failures
  ├── UnauthorizedException (401) — Missing/invalid JWT
  ├── ForbiddenException (403) — Insufficient role
  ├── NotFoundException (404) — Entity not found
  ├── ConflictException (409) — Duplicate entry (email, etc.)
  ├── UnprocessableEntityException (422) — Business rule violation
  └── InternalServerErrorException (500) — Unexpected failures

Global Exception Filter:
  ├── Catches all unhandled exceptions
  ├── Formats to standard { statusCode, message, error } shape
  └── Logs stack trace in development
```

16.2 Frontend Error Handling

```
ApiClient Error Flow:
  HTTP Response
    ├── 200–299 → Parse JSON → Return data
    ├── 401 → Attempt token refresh
    │         ├── Refresh success → Retry original request
    │         └── Refresh fail → Force logout, redirect to login
    ├── 400–499 → Show user-facing error message
    └── 500+ → Show generic "Something went wrong" message
```

---

17. VAT Calculation Logic

```
VAT Rate: 5% (UAE standard)

Price Display:
  All prices shown are VAT-inclusive

VAT Calculation for Orders:
  vatAmount = subtotal × (5 / 105)  // Extract VAT from inclusive price

  OR (if prices stored exclusive):
  vatAmount = subtotal × 0.05
  total = subtotal + vatAmount

Admin VAT Report:
  totalTaxableAmount = SUM(order.subtotal) for period
  totalVatCollected = SUM(order.vat_amount) for period
  effectiveRate = totalVatCollected / totalTaxableAmount × 100
```

End of Low Level Design Document